A COMPLETE, VISUAL GUIDE

Decision Trees Classification & Regression

From "20 questions" intuition to entropy, Gini, information gain, variance reduction, pruning, and the exact greedy algorithm that builds every tree — explained from scratch, with pictures for every idea.

🌳 Classification & Regression | 🧮 Entropy & Gini explained | ✂ Pruning & overfitting | 🐍 Python examples

A dark-mode tutorial · Easy language, real depth

CONTENTS

What's Inside

01	What Is a Decision Tree? The "20 Questions" Idea	Intuition
02	Anatomy of a Tree: Nodes, Splits, Leaves	Basics
03	How Does a Tree Pick the Best Split?	Core Concept
04	Entropy — Measuring Disorder	Math
05	Information Gain — Choosing the Best Question	Math
06	Gini Impurity — the Faster Alternative	Math
07	Worked Example: Building a Tree by Hand	Worked Example
08	The Greedy Algorithm, From Scratch	Algorithm
09	Regression Trees: Predicting Numbers	Regression
10	Visualizing Decision Boundaries	Visualization
11	Overfitting, Stopping Rules & Pruning	Pro-Level
12	Feature Importance & Other Pro Details	Pro-Level
13	Full Python Example with scikit-learn	Code
14	Pros, Cons, and When to Use Trees	Summary
15	One-Page Cheat Sheet	Recap

HOW TO READ THIS

Every math section starts with plain-English intuition and ends with a picture. If you just want the "gist," read the **Intuition** boxes and look at the charts — the equations are there whenever you're ready to go deeper.

SECTION 01

What Is a Decision Tree? The "20 Questions" Idea

The most intuitive model in all of machine learning.

You've played "20 Questions" before: someone thinks of an object, and you ask yes/no questions — "Is it alive?", "Is it bigger than a car?" — narrowing down the possibilities until you can guess correctly. A **Decision Tree** is exactly this game, learned automatically from data.

It asks a series of simple yes/no (or "is feature ≤ threshold?") questions about your data, one at a time, and each answer leads down a different branch — until it reaches a final answer: a **class label** (for classification) or a **number** (for regression).

INTUITION

Think of a doctor diagnosing a patient: "Is your temperature above 38°C?" → yes → "Do you have a cough?" → yes → "Likely flu." Every question narrows things down. A decision tree does the same thing, except it automatically figures out *which* questions to ask, and in *what order*, by looking at data.

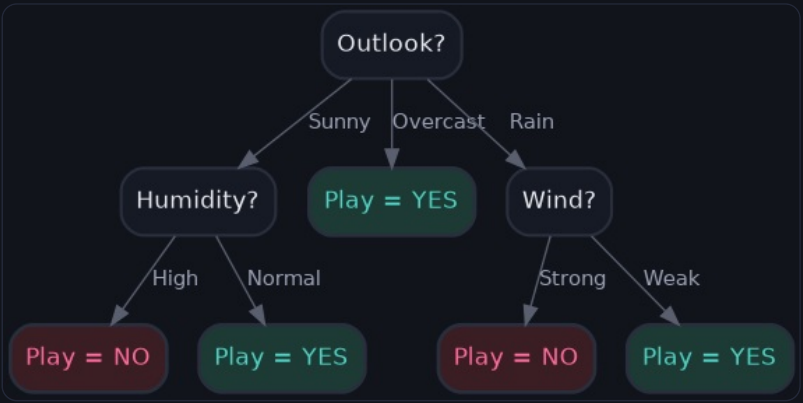


Fig 1 — A tiny decision tree deciding whether to "Play" based on weather conditions. Each internal box asks a question; each colored leaf gives a final answer.

Classification vs. Regression trees

Classification Tree

Predicts a **category** (e.g. Spam/Not Spam, Yes/No, Species A/B/C). Each leaf outputs the majority class of the training points that landed there.

Regression Tree

Predicts a **number** (e.g. house price, temperature). Each leaf outputs the average value of the training points that landed there.

Remarkably, both variants use almost the exact same algorithm — the only real difference is *how* "*good*" *a split is measured* (Sections 4–6 and 9), and *what a leaf predicts*. Everything else — how the tree grows, when it stops, how it's pruned — is shared.

SECTION 02

Anatomy of a Tree: Nodes, Splits, Leaves

The vocabulary you need before the math makes sense.

Term	Meaning
Root node	The very first question, at the top of the tree — asked to every single data point.
Internal node / split	Any question node with children — tests one feature against a threshold (e.g. "Age ≤ 30?").
Branch / edge	The path taken depending on the answer (yes/no, or which category).
Leaf node	A terminal node with no further questions — outputs the final prediction.
Depth	How many questions deep a leaf is from the root. A "depth-3" tree asks at most 3 questions per prediction.

For numeric features, splits are almost always of the form `feature ≤ threshold` — this is called an **axis-aligned split**, because geometrically it's a straight vertical or horizontal cut through feature space.

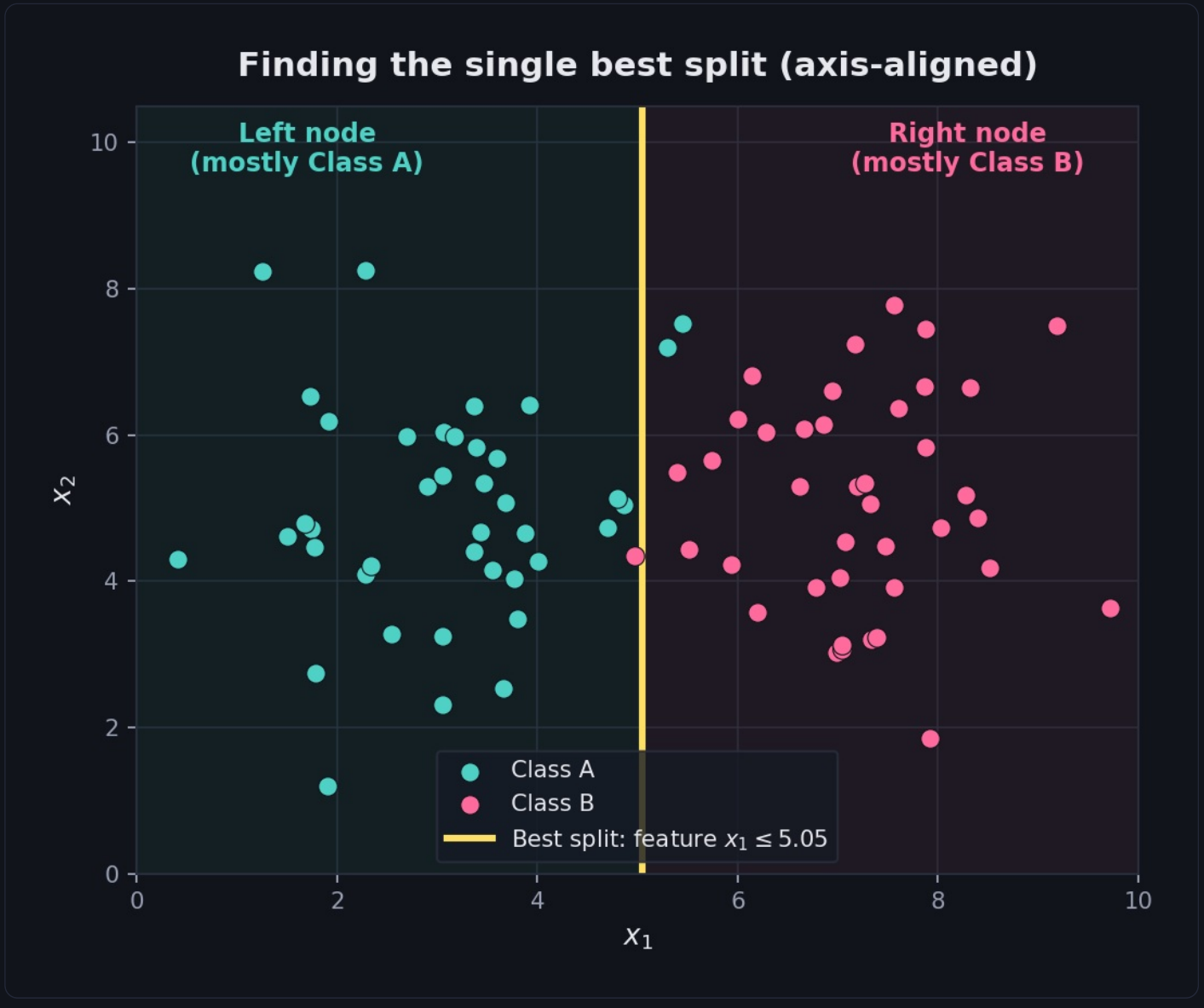


Fig 2 — A single split on feature x_1 divides the 2D plane into two rectangular regions, one mostly Class A, one mostly Class B. Every split a tree makes is a cut exactly like this.

INTUITION

A full decision tree is just this cutting process repeated recursively: cut the space in two, then cut each of those pieces again, and again, until each little region is (almost) pure — containing mostly one class, or very similar

numeric values.

How prediction works, once the tree is built

To predict a brand-new data point, you simply start at the root and answer each question using that point's feature values, following the corresponding branch, until you land on a leaf. Whatever that leaf says — that's the prediction. This makes trees extremely fast at prediction time: just a handful of comparisons.

SECTION 03

How Does a Tree Pick the Best Split?

This is the one real decision the algorithm has to make, over and over.

At every node, the tree faces a choice: which feature should it ask about, and at what threshold? To answer this, it needs a way to score "how good" a candidate split is. The scoring idea is always the same shape:

INTUITION

A good split should take a messy, mixed-up group of data points and separate it into two groups that are each *more uniform* than before — more "pure." If a split perfectly separates all class-A points from all class-B points, that's a fantastic split. If a split does nothing to separate them, it's useless.

To measure "how mixed up" a group is, we need an **impurity measure**. For classification, the two most common are **Entropy** and **Gini impurity**. For regression, we use **variance** (or mean squared error). The tree tries many candidate splits (every feature, many thresholds) and picks whichever one reduces impurity the most.

1. Measure the impurity of the current node (before splitting).
2. For every possible split (feature + threshold), compute the impurity of the two resulting child groups.
3. Compute the **weighted average impurity** of the children — splits producing purer children score better.
4. Pick the split that reduces impurity the most (equivalently: maximizes "information gain"). Repeat inside each child.

WEIGHTED IMPURITY OF A SPLIT (GENERAL FORM)

$$Impurity_{split} = \frac{n_{left}}{n} Impurity(left) + \frac{n_{right}}{n} Impurity(right)$$

Here `n` is the number of points in the parent node, and `n_left, n_right` are the counts sent to each child. This weighting matters: a split that makes one tiny, perfectly pure group and leaves a huge messy group isn't actually very useful.

SECTION 04

Entropy — Measuring Disorder

Borrowed from information theory, and surprisingly intuitive.

Entropy measures how "surprising" or "mixed" a group of labels is. If a group is all one class, there's no surprise — entropy is 0. If a group is split 50/50 between two classes, that's maximum uncertainty — entropy is at its highest.

ENTROPY OF A SET S WITH C CLASSES

$$H(S) = -\sum_{i=1}^{c} p_i \log_2 (p_i)$$

Here p_i is the proportion of class *i* in the set. For a binary (two-class) problem this simplifies to:

$$H(S) = -p_{yes} \log_2 p_{yes} - p_{no} \log_2 p_{no}$$

WORKED NUMERIC EXAMPLE

Suppose a node has 10 examples: 6 "Yes" and 4 "No". Then p(yes)=0.6, p(no)=0.4.

H = −0.6·log₂(0.6) − 0.4·log₂(0.4) = −0.6·(−0.737) − 0.4·(−1.322) = 0.442 + 0.529 = **0.971 bits**.

Compare: a perfectly pure node (10 Yes, 0 No) gives H = 0. A perfectly balanced node (5 Yes, 5 No) gives the maximum, H = 1 bit.

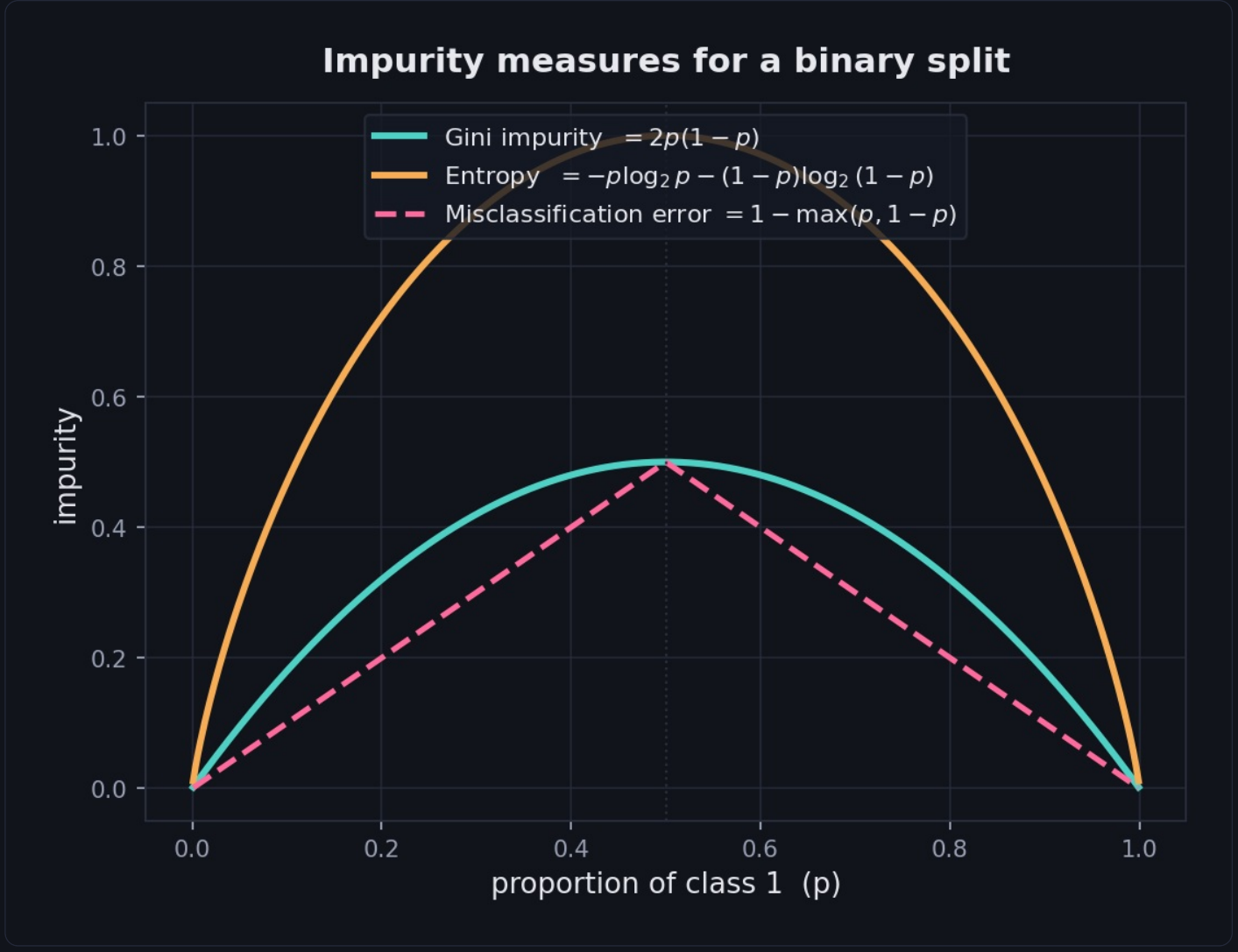


Fig 3 — Entropy (orange) peaks at 1.0 when classes are perfectly balanced (p=0.5) and hits 0 when a group is pure. Gini (teal) has the same shape but a lower peak (0.5) and is cheaper to compute (no logarithms).

WHY "BITS"?

Entropy in base-2 log is measured in bits — the same unit used in information theory to mean "how many yes/no questions, on average, would you need to correctly identify a random item's class." A pure group needs 0 questions (you already know the answer). A 50/50 group needs exactly 1 well-chosen question on average.

SECTION 05

Information Gain — Choosing the Best Question

Turning "entropy before and after" into a single score to maximize.

Information Gain (IG) asks: "how much does entropy drop after I make this split?" A big drop means the split was very informative — it did a lot of work separating the classes. A drop near zero means the split barely helped.

INFORMATION GAIN FROM SPLITTING ON ATTRIBUTE A

$$IG(S, A) = H(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} H(S_v)$$

Here `H(S)` is the entropy before splitting, and the sum computes the weighted average entropy of each child group `Sᵥ` after splitting on attribute A. The tree simply tries every candidate feature/threshold and keeps whichever gives the **highest information gain**.

CONTINUING THE WORKED EXAMPLE

Say our 10-example node (6 Yes, 4 No, H=0.971) is split by "Outlook":

- Sunny (5 examples: 2 Yes, 3 No) → H = 0.971
- Overcast (2 examples: 2 Yes, 0 No) → H = 0
- Rain (3 examples: 2 Yes, 1 No) → H = 0.918

Weighted child entropy = (5/10)(0.971) + (2/10)(0) + (3/10)(0.918) = 0.485 + 0 + 0.275 = 0.761

Information Gain = 0.971 − 0.761 = 0.210 bits. The tree would compare this to the IG of splitting on other features (Humidity, Wind, Temperature) and pick whichever feature gives the largest gain as the root question.

A SUBTLE BIAS TO KNOW ABOUT

Plain information gain is biased toward features with many distinct values (e.g. a "Customer ID" column would look artificially great, since splitting on it creates many tiny, pure groups). The fix used by some algorithms (like C4.5) is **Gain Ratio**, which divides IG by the intrinsic information of the split itself, penalizing splits that create too many small branches.

SECTION 06

Gini Impurity — the Faster Alternative

What scikit-learn actually uses by default.

Gini impurity measures something wonderfully concrete: **if you picked a random point from this group and randomly guessed its label based on the group's class proportions, how often would you be wrong?**

GINI IMPURITY OF A SET S WITH C CLASSES

$$Gini(S) = 1 - \sum_{i=1}^{c} p_i^2$$

SAME WORKED EXAMPLE, WITH GINI

Node with 6 Yes, 4 No → p(yes)=0.6, p(no)=0.4.

Gini = 1 − (0.6² + 0.4²) = 1 − (0.36 + 0.16) = 1 − 0.52 = **0.48**

Compare to entropy's 0.971 for the same node — Gini uses a different scale but tells basically the same story: this node is fairly mixed.

Entropy vs. Gini — does it matter which you pick?

	Entropy	Gini impurity
Formula	Uses $\log_2$ — slightly slower to compute	Uses only squares — faster to compute
Range (binary)	0 to 1	0 to 0.5
Sensitivity	Slightly more sensitive to changes in class balance	Slightly favors isolating the most frequent class
Practical difference	In practice, the two almost always produce very similar trees. Gini is the scikit-learn default mainly for speed.	

INTUITION, SIDE BY SIDE

Both entropy and Gini are just different mathematical ways of answering "how mixed is this bag of labeled marbles?" — both are exactly 0 when the bag is one color, and both peak when the bag is perfectly balanced. The shapes of their curves (see Fig 3, previous section) are nearly identical.

SECTION 07

Worked Example: Building a Tree by Hand

Let's build one complete small tree, start to finish.

Classic dataset: should we "Play Tennis" given the weather? 14 historical days, 4 features (Outlook, Temperature, Humidity, Wind), label = Play (Yes/No). 9 days were Yes, 5 were No.

1. **Root entropy:** $H(S) = -(9/14)\log_2(9/14) - (5/14)\log_2(5/14) = 0.940$ bits.

2. **Try every feature.** Compute information gain for Outlook, Temperature, Humidity, and Wind exactly as in Section 5. It turns out **Outlook gives the highest IG (≈0.246)** — more than Humidity (≈0.151), Wind (≈0.048), or Temperature (≈0.029).

3. **Outlook becomes the root.** It has three branches: Sunny, Overcast, Rain. The Overcast branch turns out to be *pure* (always Yes) — it immediately becomes a leaf, no more questions needed.

4. **Recurse into Sunny and Rain.** Within the Sunny subset, the algorithm again tries every remaining feature and finds that **Humidity** perfectly separates the remaining days. Within Rain, **Wind** perfectly separates them.

5. **Stop.** Every branch now ends in a pure leaf — no data point is misclassified by the final tree, shown in Figure 1 back in Section 1.

THE KEY HABIT TO NOTICE

Notice the process is entirely **recursive** and **greedy**: at each node, look only at the data that arrived there, pick the single best question for that data, split, and repeat independently on each child. The tree never "looks ahead" or reconsiders earlier splits — this is both its biggest strength (fast, simple) and its biggest weakness (can miss globally better trees). More in Section 8.

SECTION 08

The Greedy Algorithm, From Scratch

Turning the worked example into pseudocode you could actually implement.

```
function BUILD_TREE(dataset D, depth):
    # Stopping conditions
    if D is pure (all same label / all same value):
        return Leaf(prediction = majority_class_or_mean(D))
    if depth == max_depth or |D| < min_samples_split:
        return Leaf(prediction = majority_class_or_mean(D))

    # Find the single best split across ALL features and thresholds
    best_gain = -infinity
    for each feature f in D:
        for each candidate threshold t (typically midpoints between
                                        sorted unique values of f):
            D_left  = { rows in D where f <= t }
            D_right = { rows in D where f >  t }
            gain = impurity(D) - weighted_impurity(D_left, D_right)
            if gain > best_gain:
                best_gain, best_feature, best_threshold = gain, f, t

    if best_gain <= 0:                    # no useful split found
        return Leaf(prediction = majority_class_or_mean(D))

    # Recurse — this is what makes it a TREE, not just one split
    left_subtree  = BUILD_TREE(D_left,  depth + 1)
    right_subtree = BUILD_TREE(D_right, depth + 1)
    return Node(best_feature, best_threshold, left_subtree, right_subtree)
```

Every real implementation (CART, ID3, C4.5, and scikit-learn's `DecisionTreeClassifier` / `Regressor`) follows this same skeleton. The differences between them are mostly in:

- ▸ **Which impurity measure** is used (entropy, Gini, gain ratio, MSE/variance...)
- ▸ **How splits are searched** (CART only ever makes binary splits; ID3 can split categorical features into many branches at once)
- ▸ **Stopping / pruning strategy** (Section 11)

WHY "GREEDY" IS WORTH REMEMBERING

At every node the algorithm commits to the locally-best split and never revisits that decision, even if a slightly worse first split would have enabled much better splits later on. Finding the globally optimal tree is NP-hard, so every practical algorithm accepts this greedy approximation as a good-enough trade-off for speed.

SECTION 09

Regression Trees: Predicting Numbers

Same tree, same recursion — just a different scoring rule and a different leaf output.

For regression, "impurity" is replaced with **variance** (equivalently, mean squared error around the group's own mean). A "pure" leaf now means all its values are close together, rather than all the same class.

MEAN SQUARED ERROR OF A GROUP S

$$MSE(S) = \frac{1}{|S|} \sum_{i \in S} (y_i - \bar{y}_S)^2$$

where `ȳ_S` is the mean of the target values in S. A split is scored by how much it reduces this:

MSE REDUCTION FROM A SPLIT (REGRESSION'S "INFORMATION GAIN")

$$\Delta MSE = MSE(S) - \left(\frac{n_{left}}{n} MSE(S_{left}) + \frac{n_{right}}{n} MSE(S_{right}) \right)$$

And each leaf, instead of a majority vote, predicts the simple average of the training targets that land there:

$$\hat{y}_{leaf} = \bar{y}_S = \frac{1}{|S|} \sum_{i \in S} y_i$$

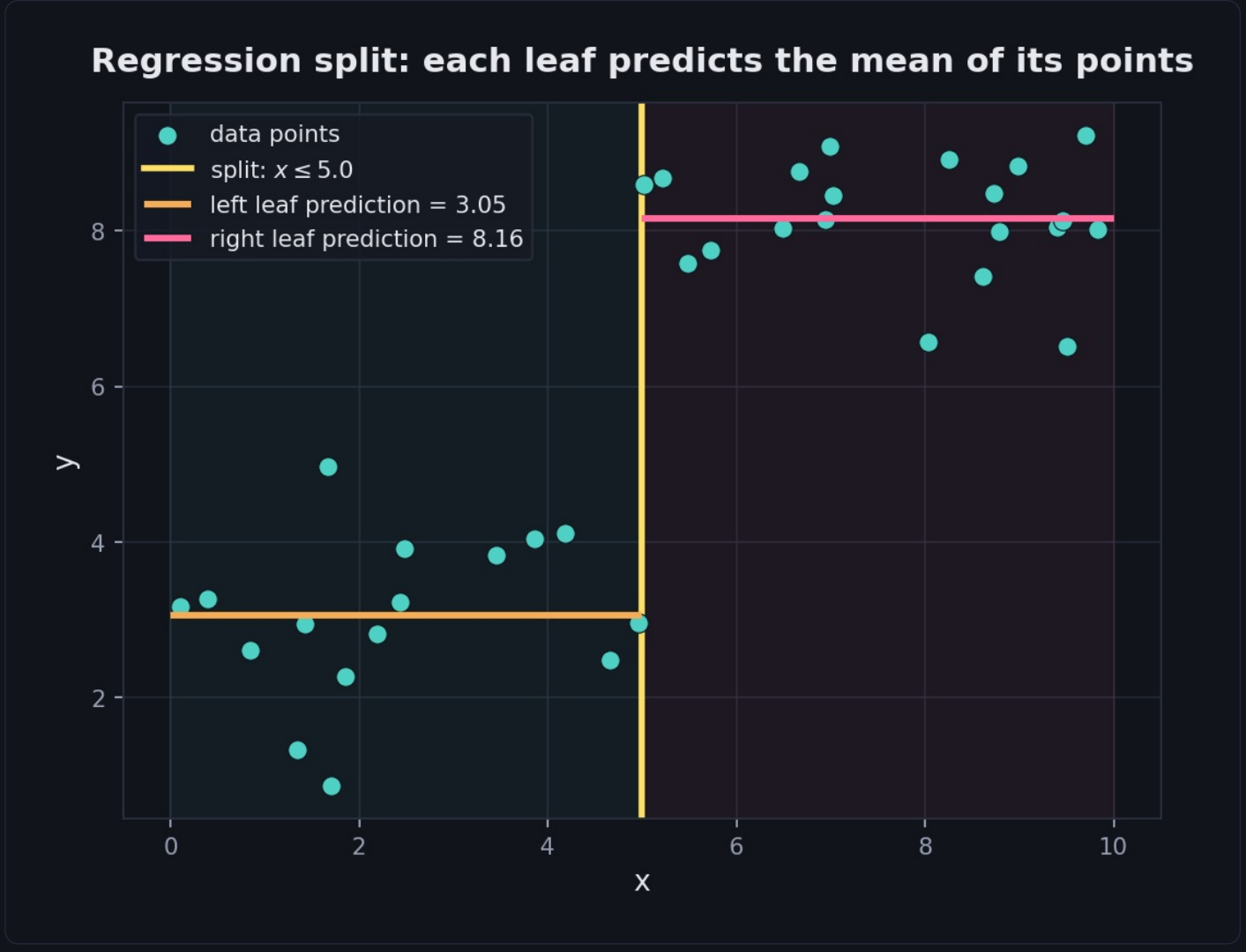


Fig 4 — A regression split chooses the threshold that makes each side's values as tightly clustered as possible around their own mean.

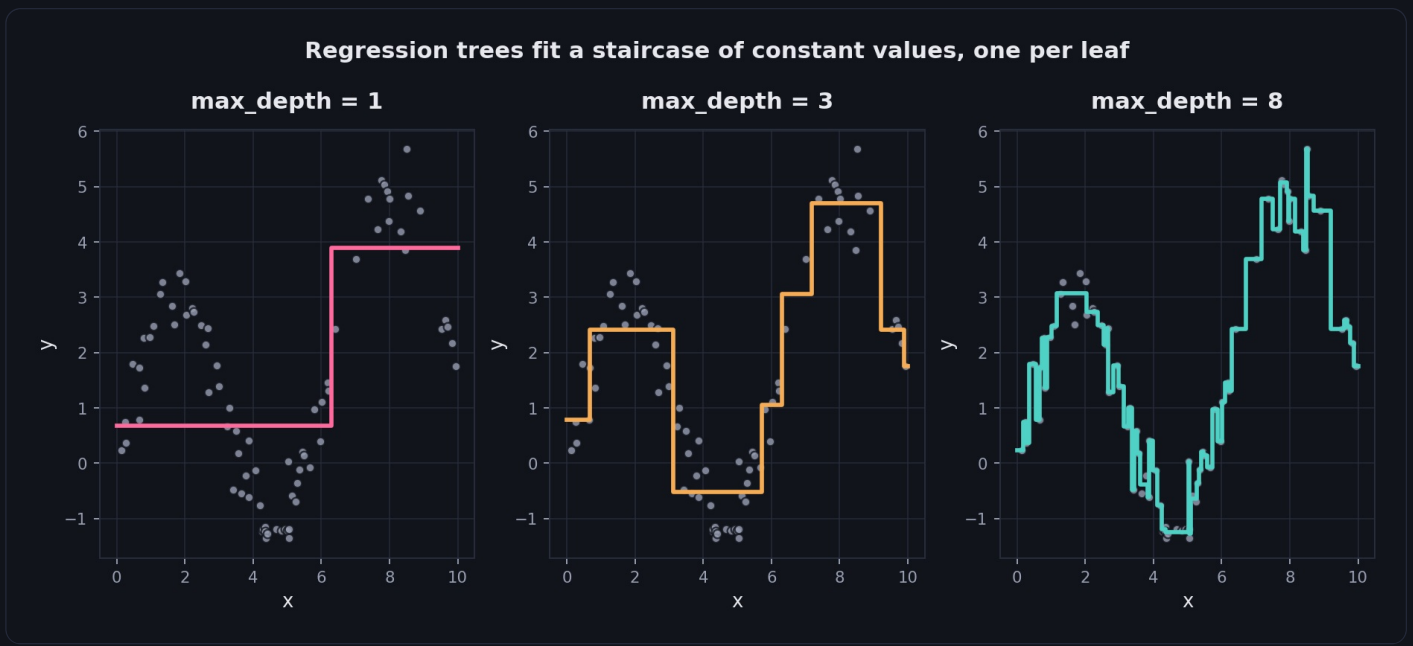


Fig 5 — Because every leaf outputs one constant number, a regression tree's prediction is always a "staircase" (step function) — never a smooth curve. Deeper trees create more, smaller steps.

INTUITION

A regression tree can never draw a smooth diagonal line — its predictions are always flat within a region, because every leaf just returns one fixed number. This is a fundamental structural property to remember: trees approximate curves with staircases, not slopes.

SECTION 10

Visualizing Decision Boundaries

Watching complexity grow as depth increases.

Because every split is a straight axis-aligned cut, a decision tree's overall decision boundary is built entirely out of rectangles — stacked and nested inside one another. The more splits (the deeper the tree), the more rectangles, and the finer the boundary can approximate a curved true pattern.

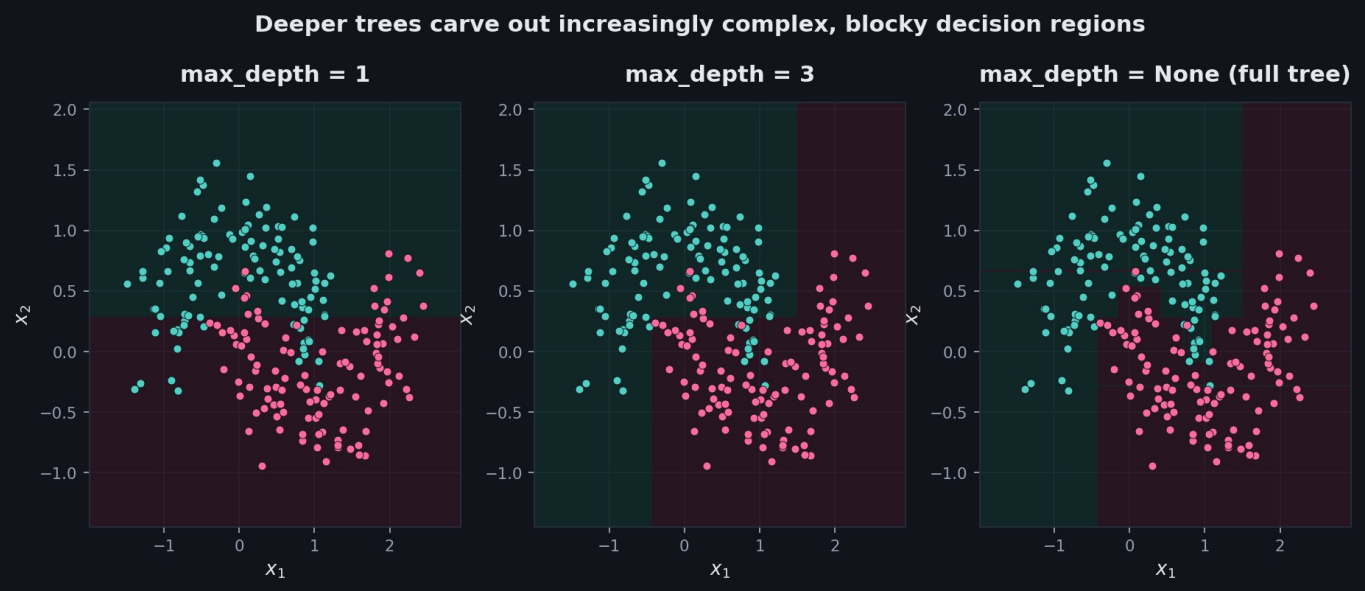


Fig 6 — At depth 1, the tree can only make one cut (badly underfitting this curved "moons" dataset). At depth 3 it captures the rough shape. Left unrestricted, it carves out a highly detailed, blocky boundary that hugs every training point — including noise.

INTUITION

Notice the unrestricted tree's boundary has odd little zigzags chasing individual noisy points — that's not a good sign. This directly previews the overfitting problem covered next: an unconstrained tree is powerful enough to memorize the training set, which usually hurts its performance on new data.

SECTION 11

Overfitting, Stopping Rules & Pruning

The single most important practical skill for using trees well.

A decision tree, left completely unconstrained, will keep splitting until every leaf is perfectly pure — which usually means it has memorized noise along with the real signal. This is **overfitting**: training accuracy near 100%, but poor performance on new data.

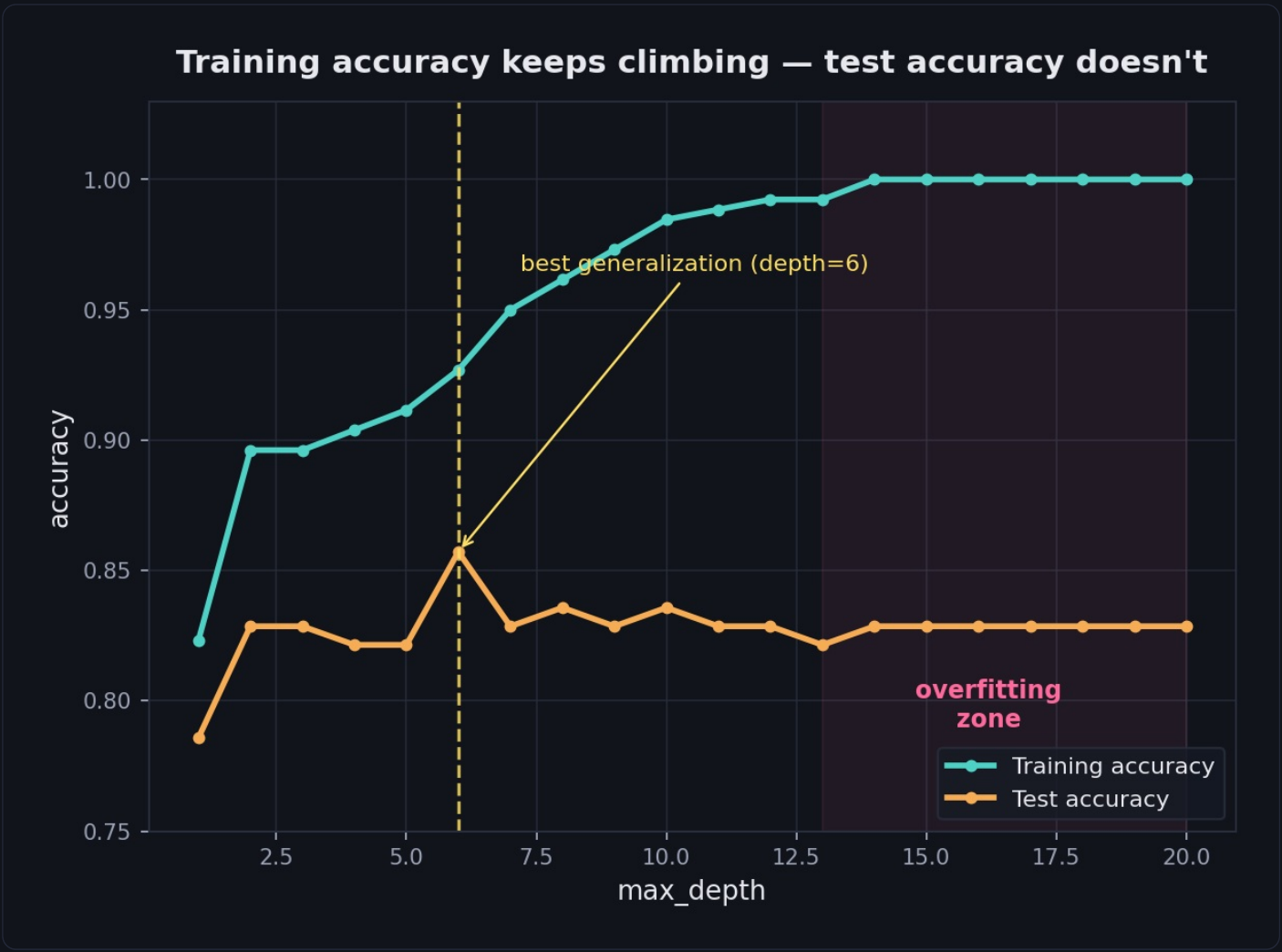


Fig 7 — As max_depth increases, training accuracy keeps climbing toward 100% — but test accuracy peaks early, then degrades. The gap between the two curves is the overfitting.

Strategy 1: Pre-pruning (stop growing early)

Hyperparameter	What it controls
`max_depth`	Hard limit on how many questions deep the tree can go
`min_samples_split`	A node needs at least this many samples before it's allowed to split further
`min_samples_leaf`	Every leaf must contain at least this many samples — prevents tiny, noise-driven leaves
`max_leaf_nodes`	Hard cap on the total number of leaves in the tree
`min_impurity_decrease`	A split must reduce impurity by at least this much, or it's rejected

Strategy 2: Post-pruning (grow fully, then trim back)

Grow the tree to its full, overfit size, then remove branches that don't meaningfully help on validation data. The most

principled version is **Cost-Complexity Pruning** (used by CART and scikit-learn's `ccp_alpha`):

COST-COMPLEXITY CRITERION FOR A SUBTREE T

$$R_\alpha(T) = R(T) + \alpha |T|$$

Here `R(T)` is the tree's error on training data, `|T|` is the number of leaves, and `α` (alpha) is a penalty per leaf. Larger α favors smaller, simpler trees. Sweeping α from 0 upward produces a sequence of nested subtrees; you pick the α that performs best on held-out/cross-validation data.



Fig 8 — As the pruning penalty α increases, the tree shrinks. Training accuracy drops smoothly, but test accuracy can actually improve for a while before both eventually decline together.

PRACTICAL TIP

In scikit-learn, start simple: set a reasonable `max_depth` (try 3–10) and `min_samples_leaf` (try 5–50 depending on dataset size), then tune with cross-validation. Reach for `ccp_alpha` pruning when you want a more principled, data-driven way to pick tree size.

SECTION 12

Feature Importance & Other Pro Details

Squeezing extra insight out of a trained tree.

Feature importance

Every time a feature is used for a split, it produces some impurity reduction (weighted by how many samples pass through that node). Summing this across every split that uses a given feature, across the whole tree, gives that feature's **importance score** — a simple, built-in way to see which inputs the tree relied on most.

INTUITION

A feature used near the root, splitting large groups of data with a big impurity drop, gets a high importance score. A feature that's only used deep in one rare branch, splitting a handful of points, barely moves the needle.

Handling categorical features

CART-style trees (including scikit-learn) convert categorical variables into a series of binary questions ("is category == X?") or require one-hot encoding beforehand. Some other implementations (like the original ID3/C4.5) can split a categorical feature into one branch per category directly.

Handling missing values

Production-grade tree implementations (e.g. XGBoost, LightGBM) learn a default direction for missing values at each split, based on which side minimizes loss during training — so a missing value doesn't crash prediction, it just follows a learned default path.

Instability — a tree's Achilles' heel

SOMETHING EVERY PRACTITIONER LEARNS THE HARD WAY

Small changes in training data can lead to a completely different first split — which cascades into a completely different tree shape. This **high variance** is the core motivation behind ensemble methods like **Random Forests** (many trees on bootstrapped samples, averaged) and **Gradient Boosted Trees** (many small trees, each correcting the last one's errors) — both build directly on everything in this guide.

SECTION 13

Full Python Example with scikit-learn

From theory to working code, for both tasks.

Classification

```
# 1. Imports
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.datasets import load_iris

# 2. Load data & split
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=0)

# 3. Tune depth / leaf size / pruning together via cross-validation
param_grid = {"max_depth": [2,3,4,5,None], "min_samples_leaf": [1,5,10],
              "criterion": ["gini","entropy"]}
grid = GridSearchCV(DecisionTreeClassifier(random_state=0), param_grid, cv=5)
grid.fit(X_train, y_train)
best_tree = grid.best_estimator_
print("Best params:", grid.best_params_)
print("Test accuracy:", best_tree.score(X_test, y_test))

# 4. Inspect feature importance
for name, importance in zip(load_iris().feature_names, best_tree.feature_importances_):
    print(f"{name}: {importance:.3f}")
```

Regression

```
# Same shape, different estimator + scoring
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import GridSearchCV

param_grid = {"max_depth": [2,3,4,5,8], "min_samples_leaf": [1,5,10]}
grid = GridSearchCV(DecisionTreeRegressor(random_state=0), param_grid, cv=5,
                    scoring="neg_mean_squared_error")
grid.fit(X_train, y_train)
best_tree = grid.best_estimator_
print("Test R^2:", best_tree.score(X_test, y_test))
```

NO FEATURE SCALING NEEDED

Unlike SVR or k-NN, decision trees only ever compare a feature to a threshold — they never compute distances between features. That means **you don't need to standardize or normalize your features** before fitting a tree.

SECTION 14

Pros, Cons, and When to Use Trees

✓ Strengths

- Easy to understand and visualize — you can literally read the decision logic
- No feature scaling required
- Handles both numeric and categorical features naturally
- Captures non-linear relationships and feature interactions automatically
- Built-in feature importance
- Fast to train and predict on small/medium data

✗ Weaknesses

- High variance — small data changes can produce very different trees
- Prone to overfitting without careful tuning/pruning
- Axis-aligned splits struggle with diagonal decision boundaries
- Regression predictions are step functions, never smooth
- Usually outperformed in raw accuracy by ensembles (Random Forest, Gradient Boosting)

When to reach for a single decision tree

- ▸ You need an **interpretable** model you can explain to non-technical stakeholders
- ▸ Your dataset mixes numeric and categorical features without much preprocessing effort
- ▸ You want a fast baseline before trying ensembles
- ▸ You're building an ensemble anyway (Random Forest / Gradient Boosting) — trees are the "base learner"

For maximum predictive accuracy on tabular data, a single tree is rarely the final answer — but it is the essential building block that Random Forests, Gradient Boosting, and XGBoost/LightGBM all combine by the hundreds or thousands. Understanding one tree deeply is what makes those models make sense.

SECTION 15

One-Page Cheat Sheet

The core idea

Recursively ask the single best yes/no question that splits the data into the purest possible groups, until a stopping condition is hit. Predict by following questions down to a leaf.

Concept	Formula	Used for
Entropy	$-\Sigma\ p_i \log_2(p_i)$	Classification impurity
Gini impurity	$1 - \Sigma\ p_i^2$	Classification impurity (sklearn default)
Information Gain	H(parent) − weighted avg H(children)	Scoring a candidate split
MSE / Variance	$\text{mean}(\ (y_i - \bar{y})^2\ )$	Regression impurity
Cost-complexity	$R(T) + \alpha \cdot \lvert T \rvert$	Post-pruning

Classification vs Regression, at a glance

Same recursive splitting algorithm. Classification uses entropy/Gini and predicts the majority class per leaf. Regression uses variance/MSE and predicts the mean value per leaf.

Fighting overfitting

Pre-pruning: limit `max_depth`, `min_samples_leaf`, `min_samples_split`. Post-pruning: grow fully, then trim with `ccp_alpha` chosen via cross-validation. Always check the train-vs-test accuracy gap.

The five-step mental model

1) Start with all data at the root → 2) try every feature/threshold, score by impurity reduction → 3) pick the best split → 4) recurse on each child → 5) stop by depth/size rule or pruning, then predict via majority-vote or mean at each leaf.

— End of tutorial —